

UniDec Antibody Light-Chain Ratio Toolkit — User Manual

Tools for quantifying the abundance ratio of a **test antibody light chain** (≈ 23412 Da) to a co-analyzed **internal-standard light chain** (≈ 23658 Da) from ensemble top-down native-MS `.raw` files (Thermo Q Exactive Plus), across a concentration series.

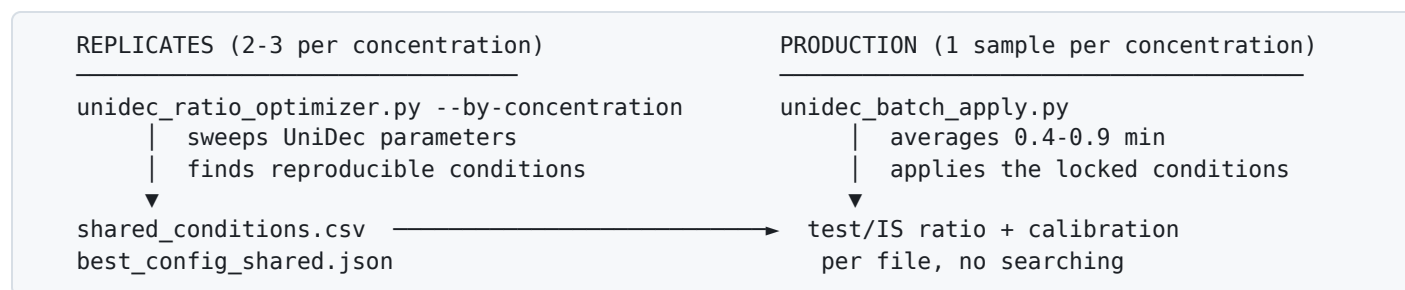
The measurement is **relative**: the reported number is the test/IS **ratio** (dimensionless, normalization-independent). Peak areas are in arbitrary UniDec-normalized units and are only meaningful relative to each other — not absolute ion counts.

Contents

1. The workflow in one picture
 2. Requirements & setup
 3. Running on Windows — command-line basics
 4. Filename convention (concentration tokens)
 5. Script A — `unidec_ratio_optimizer.py` (find conditions)
 6. Script B — `unidec_batch_apply.py` (apply conditions, `.raw` $\rightarrow$ ratio)
 7. Script C — `mz_ratio_direct.py` (deconvolution-free cross-check)
 8. Recommended end-to-end workflow
 9. Interpreting the key outputs
 10. Troubleshooting
-

1. The workflow in one picture

There are two phases, and two scripts for them:



- **Phase 1 (once):** run the **optimizer** on replicate data to find one set of UniDec conditions that gives reproducible ratios. It writes `shared_conditions.csv` and `best_config_shared.json`.
- **Phase 2 (routine):** feed that config to **batch-apply**, which averages each `.raw` over the elution window and deconvolves it with the fixed conditions — so future runs need only one sample per concentration.

`mz_ratio_direct.py` is an independent, model-free cross-check.

2. Requirements & setup

- **Windows** (Thermo `.raw` reading is Windows-native), Python 3.9+.
 - **UniDec 8.x** installed in the same Python environment (`pip install unidec`). This manual is verified against **UniDec 8.1.3**.
 - Python packages: `numpy` , `scipy` (peak-fit quality), `matplotlib` (plots). `pip install unidec numpy scipy matplotlib`
 - **Both** `unidec_ratio_optimizer.py` and `unidec_batch_apply.py` must sit in the **same folder** — the batch script imports the optimizer for its shared deconvolution code.
-

3. Running on Windows — command-line basics

Run in **Command Prompt** or **PowerShell** (not an IDE "Run" button — that can duplicate the script name and cause `unrecognized arguments` errors).

- Put the **whole command on one line**.
- **Quote any path with spaces** in double quotes: `--data "D:\Internal Data\...\Test"` .
- **Do not** use `\` to continue lines (that is Unix syntax; on Windows the `\` becomes a stray argument). Use `^` (cmd) or ``` (PowerShell) if you must.
- **Do not** end a quoted path with a trailing backslash before the quote (`"...\Test\"` escapes the quote).

Tip: `cd` into the script folder first, then you can call the script by name:

```
cd C:\path\to\unidec_scripts
python unidec_batch_apply.py ...
```

4. Filename convention (concentration tokens)

Grouping into concentrations is done from a token in each filename, written in scientific notation with **p** as the **decimal point**:

Filename contains	Parsed concentration
<code>1p00e-4</code>	1.0×10^{-4} µg/mL
<code>2e-4</code>	2×10^{-4} µg/mL
<code>5p00e-5</code>	5×10^{-5} µg/mL

Files sharing a token are treated as replicates of the same concentration. Files with **no** token are reported and excluded from concentration grouping. Override the pattern with `--conc-pattern "<regex>"` if your naming differs.

5. Script A — `unidec_ratio_optimizer.py` (find conditions)

Sweeps UniDec deconvolution parameters over your replicate data to find good conditions. It runs in one of four **modes** (pick one flag).

Input

- `--data <folder>` **(required)** — folder of replicate `.raw` / `.mzML` / `.mzXML` files. (This script feeds files straight to UniDec's default import; it does **not** do the 0.4–0.9 min averaging — that lives in the batch script.)

Modes

Flag	Objective	Use it for
<i>(none)</i>	One shared parameter set maximizing mean peak-fit quality	A single best set of conditions
<code>--by-concentration</code>	Per-concentration min-CV and a shared min-CV set, side by side	Your calibration workflow (recommended)
<code>--per-file</code>	Each file optimized independently by its own quality	Diagnosing per-file behavior
<code>--cv-mode</code>	One shared set minimizing global cross-replicate %CV	Legacy objective

What gets swept

Full grid = **729** parameter sets (3^6); `--quick` = **32**.

App label	Config	Full-grid values
Peak FWHM	<code>mzsig</code>	0.3, 0.4, 0.5
Charge Smooth Width	<code>zzsig</code>	1.0, 5.0, 10.0
Point Smooth Width	<code>psig</code>	0.0, 1.0, 2.0
Beta	<code>beta</code>	0.0, 50.0, 100.0
Gaussian Smoothing	<code>smooth</code>	1.0, 2.0, 5.0
Subtract Curved width	<code>subbuff</code>	50, 100, 150

Fixed (not swept): `massbins=0.1` Da, `mzbins=1.0` Th, `subtype=1` (curved), m/z window `700–2000`, charge range `8–45`.

Options

Option	Default	Meaning
<code>--out <name></code>	<code>unidec_ratio_results</code>	Output dir base (timestamp appended)
<code>--quick</code>	off	Small 32-set grid for a first pass
<code>--by-concentration</code>	off	Per-concentration + shared analysis
<code>--per-file</code>	off	Optimize each file separately
<code>--cv-mode</code>	off	Legacy global-%CV objective

Option	Default	Meaning
<code>--quality-floor <f></code>	0.5	(by-conc) min peak quality a set must reach; 0 disables
<code>--cv-margin <f></code>	0.25	(by-conc) among sets within this fraction of best CV, pick highest quality
<code>--conc-pattern <re></code>	1p00e-4 -style	Override concentration-token regex
<code>--centroidA <Da></code>	23412.0	Test light-chain mass (ratio is A/B)
<code>--centroidB <Da></code>	23658.0	Internal-standard light-chain mass
<code>--minmz <Th> / --maxmz <Th></code>	700 / 2000	m/z window fed to UniDec
<code>--zrange <lo> <hi></code>	8 45	Charge-state range
<code>--mass-pad <Da></code>	910	Headroom around centroids for the mass grid
<code>--integration-mode {window,gaussian}</code>	window	How the mass spectrum becomes a ratio
<code>--ratio-band <lo> <hi></code>	1.8 4.5	(cv-mode only) accepted mean-ratio band
<code>--target-ratio <r></code>	3.0	(cv-mode only) fallback target if none in band
<code>--lam <w></code>	1.0	(cv-mode only) ratio-drift penalty weight
<code>--fresh-engine</code>	off	Disable engine reuse (slower; debugging only)
<code>--no-timestamp</code>	off	Do not append a timestamp to <code>--out</code>
<code>--verbose</code>	off	Verbose logging

Outputs (by mode)

- **--by-concentration**: `comparison.csv` (per-concentration vs shared CV/ratio whether params differ), `per_concentration_best.csv`, `shared_conditions.csv` (the single shared set + calibration slope/intercept/R²), `best_config_shared.json`, `best_config_by_concentration.json`, `per_file_ratios.csv`, `by_concentration_sweep_full.csv` (audit), `ratio_vs_concentration.png`.
- **default (quality)**: `quality_sweep_results.csv`, `best_config.json`, `best_per_file.csv`, `quality_vs_ratio.png`.
- **--per-file**: `per_file_optimized.csv`, `best_config_per_file.json`, `per_file_sweep_full.csv`, `per_file_ratios.png`.
- **--cv-mode**: `sweep_results.csv`, `best_config.json`, `best_per_file.csv`, `ratio_vs_cv.png`.

Example

```
python unidec_ratio_optimizer.py --data "D:\...\replicates" --out results --by-concentration --quick
python unidec_ratio_optimizer.py --data "D:\...\replicates" --out results --by-concentration
```

6. Script B — `unidec_batch_apply.py` (apply conditions, `.raw` → `ratio`)

Applies **fixed** conditions to `.raw` files — no searching. This is the production script and the full start-to-finish path.

Inputs

- `--config <file>` (**required**) — the conditions to apply:
- `best_config_shared.json` — the **complete** parameter set (recommended; includes geometry, exact reproduction).
- `shared_conditions.csv` — the **tuned** parameters only; geometry (centroids, m/z window, charge range) is filled from the CLI defaults below, so pass the same `--centroidA/--centroidB/--minmz/--maxmz/--zrange` you optimized with if you changed them.
- `--data <folder>` (**required**) — folder of `.raw` files.

What it does per `.raw`

1. **Averages** scans over the elution window (`--time-lo` .. `--time-hi` , minutes) into one **profile** spectrum via UniDec's importer.
2. **Saves** it to `averaged_spectra\<name>_avg.txt` (two-column m/z + intensity, re-importable into UniDec).
3. **Deconvolves** that averaged spectrum with the fixed conditions → test/IS ratio.

Options

Option	Default	Meaning
<code>--config <file></code>	<i>(required)</i>	<code>shared_conditions.csv</code> or <code>best_config_shared.json</code>
<code>--data <folder></code>	<i>(required)</i>	Folder of <code>.raw</code> files
<code>--out <dir></code>	<i>inside --data</i>	Output dir. Omit → timestamped folder inside the data directory . Pass a path to send results elsewhere (needed if the data folder is read-only).
<code>--time-lo <min></code>	<code>0.4</code>	Start of elution window to average
<code>--time-hi <min></code>	<code>0.9</code>	End of elution window to average
<code>--no-average</code>	off	Skip averaging; feed files as-is (for already-averaged spectra)
<code>--centroidA <Da></code>	<code>23412.0</code>	Test mass (CSV config only; ignored for JSON)
<code>--centroidB <Da></code>	<code>23658.0</code>	IS mass (CSV config only)
<code>--minmz / --maxmz</code>	<code>700 / 2000</code>	m/z window (CSV config only)
<code>--zrange <lo> <hi></code>	<code>8 45</code>	Charge range (CSV config only)
<code>--mass-pad <Da></code>	<code>910</code>	Mass-grid headroom (CSV config only)

Option	Default	Meaning
<code>--fresh-engine</code>	off	Disable engine reuse
<code>--no-timestamp</code>	off	Do not append a timestamp to <code>--out</code>
<code>--verbose</code>	off	Verbose logging

Outputs (in `<data>\unidec_batch_results_<timestamp>\` by default)

- `averaged_spectra\<name>_avg.txt` — one averaged spectrum per `.raw`.
- `batch_per_file.csv` — ratio + peak quality per file (with `averaged_spectrum` column linking to the saved spectrum).
- `batch_by_concentration.csv` — mean / SD / %CV per concentration.
- `batch_summary.txt` — the exact conditions applied + calibration slope / intercept / R^2 .
- `ratio_vs_concentration.png` — the calibration curve.

Example

```
python unidec_batch_apply.py --config "C:\...\best_config_shared.json" --data "D:\Internal Data\...\\"

```

(Results land in `D:\Internal Data\...\Test\unidec_batch_results_<timestamp>\`.)

For already-averaged files, add `--no-average`. For a different window, add `--time-lo 0.5 --time-hi 1.0`.

7. Script C — `mz_ratio_direct.py` (deconvolution-free cross-check)

Computes the A/B ratio **without deconvolution** by integrating the raw m/z signal per charge state in a "safe zone". Use it as an independent sanity check against the UniDec-based result.

Input

- `--data <folder>` **(required)** — folder of `.raw` files.

Options

Option	Default	Meaning
<code>--out <dir></code>	<code>results_direct</code>	Output directory
<code>--massA <Da></code>	<code>23412.0</code>	Test light-chain zero-charge mass
<code>--massB <Da></code>	<code>23658.0</code>	Internal-standard mass
<code>--zmin <z> / --zmax <z></code>	<code>15 / 22</code>	Charge-state "safe zone" to integrate
<code>--half-width <Th></code>	<code>1.5</code>	Integration half-window ($\pm$ m/z) around each expected peak
<code>--baseline-flanks <Th></code>	<code>3.0</code>	Width of flanking regions for local baseline

Option	Default	Meaning
<code>--verbose</code>	off	Verbose logging

Outputs

- `direct_per_file.csv` — ratio + per-charge-state totals per file.
- `direct_zprofile.csv` — mean area vs charge state (diagnostic).
- `direct_per_z_detail.csv` — full per-file per-z audit trail.
- `direct_summary.txt` — mean / SD / %CV.
- `direct_plots.png` — per-file ratios + charge-envelope shape.

Example

```
python mz_ratio_direct.py --data "D:\...\replicates" --out results_direct
```

8. Recommended end-to-end workflow

1. **First pass (fast):** find conditions on your replicate data. `python unidec_ratio_optimizer.py --data "D:\...\replicates" --out results --by-concentration --quick` Open `comparison.csv`: check the `params_differ` column (all "no" → the shared set is clearly right) and that the ratios are sensible. Eyeball the winning mass spectrum to confirm both peaks are cleanly separated.
2. **Full sweep (overnight if large):** re-run without `--quick` for the 729-set grid.
3. **Lock the conditions:** use the resulting `best_config_shared.json`.
4. **Apply to production data** (single sample per concentration), start-to-finish: `python unidec_batch_apply.py --config "C:\...\best_config_shared.json" --data "D:\...\newdata"`
5. **Cross-check (optional):** run `mz_ratio_direct.py` on the same data; if its ratio agrees with UniDec's, the value is robust to method choice.

9. Interpreting the key outputs

- **The number you report** is `ratio_A_over_B` (test/IS). It is relative and normalization-independent.
- `area_A / area_B` are arbitrary UniDec-normalized units — meaningful only *relative to each other within a file*, not as absolute abundances. Do not compare areas between files.
- **quality** (0–1) = peak-fit quality ($0.5 \cdot \text{separation} + 0.25 \cdot (\text{fitR}^2_{\text{A}} + \text{fitR}^2_{\text{B}})$). High = A and B are clean, well-separated Gaussians. Low or a `separation` near 0 means the peaks merged — a false result even if CV is low.
- **cv_percent** = cross-replicate reproducibility (lower is better). In the optimizer's default quality mode this column is descriptive, not the objective.
- **params_differ** (by-concentration `comparison.csv`) — "no" means the shared conditions match the per-concentration optimum, i.e. conditions don't drift with concentration (good; supports single-sample runs).

- **Calibration R^2** (`shared_conditions.csv` , `batch_summary.txt`) — linearity of ratio vs concentration; your headline validation metric.

⚠ Always visually inspect at least one winning mass spectrum. A low CV achieved by over-smoothing A and B into a single peak is a false win — the `separation` term guards against it, but confirm by eye.

10. Troubleshooting

Symptom	Cause / fix
<code>error: unrecognized arguments: <path fragments></code>	Path with spaces not quoted. Wrap paths in <code>"..."</code> .
<code>error: unrecognized arguments: unidec_batch_apply.py</code>	The script name is in the argument list twice — usually an IDE "Run" config that also includes the script name. Put only the flags in the IDE options box, or run from a terminal.
<code>unrecognized arguments: \</code>	You used Unix <code>\</code> line-continuations. Put the command on one line.
<code>Cannot import unidec.engine</code>	UniDec not installed in this Python env. <code>pip install unidec</code> .
<code>Could not import UniDec's ImporterFactory</code>	Averaging needs UniDec 8.x. If spectra are already averaged, add <code>--no-average</code> .
Batch run can't create the output folder	The <code>--data</code> folder is read-only (e.g. a locked network share). Pass <code>--out "C:\writable\path"</code> .
Averaged spectrum looks empty / wrong	Check the time-window units. <code>--time-lo/--time-hi</code> are retention minutes ; confirm against Xcalibur. The script falls back to whole-file averaging if the window is empty.
A concentration is skipped in by-concentration mode	It has fewer than 2 replicates (a CV needs ≥ 2).
Peaks merged / ratio implausible	Over-smoothing. Lower <code>zssig</code> / <code>beta</code> / <code>smooth</code> ; confirm <code>mzsig</code> matches the real peak width (use UniDec's auto peak-width tool on one file).

Quantification note: this toolkit measures a relative test/IS ratio from UniDec-normalized peak areas. It does not report absolute ion counts; absolute abundance would require calibrated ion counting (e.g. CDMS), which this method does not provide.